

DollarPunk

abeninati98

January 2026

Sentiment Diffusion Model over Time and a High-Dimensional Semantic Domain

1. Why a diffusion equation is useful (intuition first)

A diffusion equation is useful when you want to model how a quantity:

- **propagates** from one point to nearby points,
- **smooths** noisy observations,
- **persists** or **decays** over time.

Here the quantity is **sentiment**, and the “space” is not physical space but a **semantic domain**. The main practical outcomes are:

- **denoising**: isolated noisy sentiment spikes are reduced;
- **spillover**: sentiment can affect semantically related items (and related companies);
- **time dynamics**: you separate short-lived shocks from persistent narratives;
- **features**: you create stable signals for ranking/alerts/forecasting (e.g., in finance).

2. Your practical setting (DB already scored)

We assume you have a database of news articles indexed by i with:

- timestamp t_i ,
- **overall article sentiment** $\text{sent}_i^{\text{art}} \in [0, 1]$,
- a set of referenced companies $a \in \mathcal{A}_i$,
- **company-specific sentiment** $\text{sent}_{i,a} \in [0, 1]$,
- **company-specific relevance** $\text{rel}_{i,a} \in [0, 1]$.

(So sentiment and relevance are **referenced to the specific company**, and there is also an overall article score.)

The goal is to create a **context-aware, time-aware** company signal and optionally a **relevance-adjusted** score for each (article, company) pair.

3. What “semantic mapping” means (instead of just saying embeddings)

We introduce a mapping from text to a point in a high-dimensional semantic space:

$$\Phi : \mathcal{T} \rightarrow \mathbb{R}^n, \quad n \gg 100$$

where $\mathcal{T}$ is the set of all texts (articles, descriptions, etc.).

What does Φ map? Φ maps a text object to a vector that encodes its **meaning**. The key property is:

$$\|\Phi(t_1) - \Phi(t_2)\| \text{ small} \iff t_1 \text{ and } t_2 \text{ are semantically similar.}$$

So:

- **distance** (or cosine similarity) reflects conceptual relatedness,
- **neighborhoods** represent related topics/entities/scientific concepts,
- diffusion can be defined using these neighborhoods (“nearby in meaning”).

Each article is assigned a semantic coordinate:

$$x_i = \Phi(\text{article } i).$$

4. Pharma-specific note: what “drug pipeline” means

For pharmaceutical companies, “drug pipeline” means the set of drugs/candidates under development, usually described by:

- molecule/drug name,
- biological target (e.g., PD-1, HER2),
- therapeutic area (oncology, immunology, vaccines, ...),
- clinical phase (pre-clinical, Phase I–III, approved),
- indication (disease).

This information is a strong signal of **scientific and business proximity** between companies.

5. Do you need a fixed table of weights between companies?

To diffuse sentiment across companies you need **some notion of proximity/weights**. This can be:

- **fixed** (a static company–company table), or
- **dynamic** (computed from semantic representations), or
- **hybrid** (recommended: stable + context-sensitive).

6. How to build the company vectors v_a (what you need)

To build a company representation vector, you need **representative text** for each company a . Define:

$$v_a = \Phi(\text{text describing company } a).$$

The text can be formed by concatenating sources such as:

- business description,
- press releases,
- clinical trial summaries/abstracts,
- drug pipeline descriptions (molecules/targets/indications/phases),
- historical company-related news.

Optionally, you can keep multiple texts (pipeline vs business) and combine their vectors with weights.

7. How to build the weight matrix W between companies

We define a weighted graph over companies with adjacency matrix $W \in \mathbb{R}^{N \times N}$.

7.1. Option A: co-citation (simplest; uses only your news DB)

If companies a and b appear in the same article, increase their link:

$$(W_{\text{cocit}})_{ab} = \sum_{i: a, b \in \mathcal{A}_i} \text{rel}_{i,a} \text{rel}_{i,b} g(\text{sent}_{i,a}, \text{sent}_{i,b})$$

where $g(\cdot)$ is a nonnegative function such as $g = \frac{1}{2}(\text{sent}_{i,a} + \text{sent}_{i,b})$ or $g = |\text{sent}_{i,a} - 0.5| + |\text{sent}_{i,b} - 0.5|$ (depending on whether you want intensity around neutrality).

7.2. Option B: semantic similarity (more robust; works even without co-citation)

Use the company vectors:

$$(W_{\text{semantic}})_{ab} = \max(0, \cos(v_a, v_b))^\gamma.$$

7.3. Option C: hybrid (recommended)

$$W = \alpha W_{\text{cocit}} + (1 - \alpha) W_{\text{semantic}}, \quad \alpha \in [0, 1].$$

7.4. Practical constraints (important)

To make the model stable and scalable:

- **sparsify**: keep only top- K neighbors per company (e.g., $K = 20$ – 50);
- **remove self-loops**: set $W_{aa} = 0$;
- **normalize**: often row-normalize W or use the graph Laplacian (next section).

8. The diffusion equation (the core model)

8.1. Continuous form (conceptual)

If sentiment were a continuous field $u(x, t)$ on a semantic domain $x \in \mathbb{R}^n$:

$$\frac{\partial u}{\partial t}(x, t) = D \Delta_x u(x, t) - \lambda u(x, t) + s(x, t)$$

where:

- $D > 0$ is diffusion strength,
- Δ_x is the Laplacian over the semantic coordinates,
- $\lambda \geq 0$ is temporal decay (forgetting),
- $s(x, t)$ is a source term injecting sentiment from news.

In high dimensions ($n \gg 100$) the continuous Laplacian is usually **not** implemented directly. Instead we use a graph formulation.

8.2. Graph form (the practical version)

Let there be N companies. Define the degree matrix D_g with $(D_g)_{aa} = \sum_b W_{ab}$ and the graph Laplacian:

$$L = D_g - W.$$

Let $\mathbf{u}(t) \in \mathbb{R}^N$ be the diffused sentiment state for all companies at time t (e.g., $u_a(t)$ is the “context sentiment” impacting company a). The reaction–diffusion ODE on the company graph is:

$$\dot{\mathbf{u}}(t) = -\kappa L \mathbf{u}(t) - \lambda \mathbf{u}(t) + \mathbf{s}(t),$$

where:

- $\kappa > 0$ controls propagation across related companies,
- $\lambda \geq 0$ controls decay over time,
- $\mathbf{s}(t)$ injects news impulses into the relevant companies.

8.3. Source term with company-specific and article-level sentiment (your case)

For an article i published at time t_i and referencing company a , define the injected signal (impulse) as:

$$s_{i,a}(t) = \text{rel}_{i,a} \cdot \left[\eta \text{sent}_{i,a} + (1 - \eta) \text{sent}_i^{\text{art}} \right] \cdot \delta(t - t_i), \quad \eta \in [0, 1].$$

Interpretation:

- $\text{rel}_{i,a}$ gates how much article i matters for company a ;
- $\text{sent}_{i,a}$ is the company-specific tone (0–1);
- $\text{sent}_i^{\text{art}}$ is the overall article tone (0–1);
- η blends company-specific vs overall sentiment.

Then the total source for company a is:

$$s_a(t) = \sum_i s_{i,a}(t), \quad \text{and} \quad \mathbf{s}(t) = (s_1(t), \dots, s_N(t))^\top.$$

8.4. What the diffusion accomplishes operationally

Compared to raw aggregation (e.g., averaging sentiments per company), the diffusion dynamics:

- spreads sentiment to semantically related companies (spillover),
- smooths isolated noise via neighborhood averaging,
- retains persistent signals while decaying old information,
- yields a state variable $u_a(t)$ usable as a stable company signal.

9. Using the diffused field for relevance adjustment and ranking

9.1. Context influence

A simple definition of “contextual influence” for company a at article time t_i is:

$$I_{i,a} = u_a(t_i).$$

(Optionally, you can use a neighbor average $I_{i,a} = \sum_b \tilde{w}_{ab} u_b(t_i)$.)

9.2. Relevance adjusted score

You asked for a new adjusted relevance score that accounts for diffusion-weighted context. A direct definition is:

$$\text{rel}_{i,a}^{\text{adj}} = \text{rel}_{i,a} \left(1 + \beta I_{i,a} \right), \quad \beta \geq 0,$$

where $I_{i,a}$ is the context sentiment state. If you want an always-positive boost you can use $|I_{i,a}|$:

$$\text{rel}_{i,a}^{\text{adj}} = \text{rel}_{i,a} \left(1 + \beta |I_{i,a}| \right).$$

This implements the idea: *the same relevance becomes more “important” when the company is inside a strong sentiment narrative.*

10. Stock-level “information pressure” indicator (pharma example)

When each article references one or more pharma stocks, you want a company-level signal that summarizes the **cumulative and diffused** sentiment environment. The simplest pressure indicator is:

$$\text{Pressure}_a(t) = u_a(t).$$

Interpretation:

- if $u_a(t)$ is high, company a is under strong (positive) narrative pressure;
- diffusion means this can be driven by related-company news (e.g., same therapeutic area/pipeline proximity).

This is exactly the “spillover across companies” you want in pharma: a strong narrative around one drug class can propagate to companies with similar pipelines.

11. Minimal data requirements recap (in the order discussed)

To implement the full pipeline you need:

- from your DB: t_i , $\text{sent}_i^{\text{art}}$, and for each referenced company (a): $\text{sent}_{i,a}$ and $\text{rel}_{i,a}$;
- a way to define company proximity weights W :
 - either co-citation from the same DB,
 - or company vectors v_a from representative company text,
 - or hybrid.
- the diffusion dynamics parameters (κ, λ) and the blend η for article vs company sentiment.

12. Computational Implementation (Numerical Perspective)

This section outlines how the proposed model can be implemented in practice using standard numerical methods, without committing to a specific programming language.

12.1. Discretization in time

Time is discretized on a grid $t_0, t_1, \dots, t_T$ with step Δt . The continuous-time diffusion equation

$$\dot{\mathbf{u}}(t) = -\kappa L \mathbf{u}(t) - \lambda \mathbf{u}(t) + \mathbf{s}(t)$$

is integrated numerically. A first-order explicit scheme (forward Euler) gives:

$$\mathbf{u}^{(k+1)} = \mathbf{u}^{(k)} + \Delta t \left[-\kappa L \mathbf{u}^{(k)} - \lambda \mathbf{u}^{(k)} + \mathbf{s}^{(k)} \right],$$

where $\mathbf{u}^{(k)} \approx \mathbf{u}(t_k)$.

More stable alternatives (implicit Euler, Crank–Nicolson) can be used if κ or the graph is large.

12.2. Source term construction

At each time step t_k , the source vector $\mathbf{s}^{(k)}$ is built by aggregating all articles published in $[t_k, t_{k+1})$:

$$s_a^{(k)} = \sum_{i: t_i \in [t_k, t_{k+1})} \text{rel}_{i,a} \left[\eta \text{sent}_{i,a} + (1 - \eta) \text{sent}_i^{\text{art}} \right].$$

This corresponds to a discrete approximation of the impulse term $\delta(t - t_i)$.

12.3. Graph operators

The company similarity matrix W is stored in sparse format (after top- K sparsification). The degree matrix is defined as:

$$(D_g)_{aa} = \sum_b W_{ab}, \quad L = D_g - W.$$

Matrix–vector products $L\mathbf{u}$ dominate the computational cost and scale linearly with the number of nonzero entries in W .

12.4. Initialization and decay

The sentiment state is initialized as:

$$\mathbf{u}^{(0)} = \mathbf{0},$$

or from a historical steady state if available. The decay term $-\lambda\mathbf{u}$ ensures that in the absence of new news, the state relaxes exponentially to zero.

12.5. Online vs batch execution

- **Online setting:** update $\mathbf{u}$ sequentially as new articles arrive (streaming-compatible).
- **Batch setting:** replay historical news to compute $\mathbf{u}(t)$ over a fixed horizon (useful for backtesting and feature generation).

12.6. Outputs for downstream use

At each time step or event time t_i , the model produces:

- the diffused company sentiment state $u_a(t)$,
- contextual influence values $I_{i,a} = u_a(t_i)$,
- relevance-adjusted scores $\text{rel}_{i,a}^{\text{adj}}$.

These quantities can be stored and used directly for alerting, ranking, visualization, or as features in predictive models.

13. Practical Setup and Data Requirements

This section describes the minimal practical setup needed to implement the model, starting from a real-world database, and clarifies its applicability beyond the biopharmaceutical domain.

13.1. Applicability across sectors

The framework is *sector-agnostic*. While pharmaceutical companies were used as a motivating example (due to clear scientific and pipeline-based spillovers), the model applies equally well to:

- technology and semiconductors,
- energy and commodities,
- financial institutions,
- industrials and consumer goods.

The only requirement is that news articles can be associated with one or more companies and assigned company-specific sentiment and relevance scores.

13.2. Minimal data objects

The implementation requires three core entities:

- a universe of companies (stocks),
- a stream or archive of news articles,
- a mapping between articles and companies with sentiment annotations.

13.3. Recommended database schema

Table: articles

- `article_id` (primary key),
- `published_at` (timestamp, consistent timezone),
- `sent_article` $\in [0, 1]$ (overall article sentiment, optional),
- `title` (optional),
- `text` or `url` (optional, for semantic enrichment).

Table: companies

- `company_id` (primary key),
- `ticker`,
- `name`,
- `sector` (optional).

Table: article_company A bridge table with one row per referenced company:

- `article_id` (foreign key),
- `company_id` (foreign key),
- `sent_company` $\in [0, 1]$,
- `rel_company` $\in [0, 1]$,
- `mention_count` (optional),
- `role` (optional: main / secondary).

Primary key: (`article_id`, `company_id`).

13.4. Data quality constraints

Before running the diffusion model, the following checks are essential:

- sentiment and relevance scores clipped to $[0, 1]$,
- no duplicate rows in `article_company`,
- each article references at least one company,
- sufficient article coverage per company to avoid unstable signals,
- consistent time encoding for all articles.

13.5. First implementation step: building the weight matrix

A first operational version of the model can be implemented without semantic text processing by constructing the company weight matrix W via co-citation:

$$W_{ab} = \sum_{i: a, b \in \mathcal{A}_i} \text{rel}_{i,a} \text{rel}_{i,b} h(\text{sent}_{i,a}, \text{sent}_{i,b}),$$

where $h(\cdot)$ is a nonnegative weighting function (e.g., constant 1, mean sentiment intensity, or distance from neutrality).

This approach:

- uses only the existing annotated news database,
- requires no additional NLP infrastructure,
- is sufficient to validate the diffusion dynamics end-to-end.

13.6. Incremental extension

Once the baseline pipeline is validated, the model can be extended by:

- adding semantic company vectors to refine W ,
- introducing time-decayed or adaptive weight matrices,
- tuning diffusion and decay parameters via backtesting.

This staged approach allows the model to be deployed progressively, starting from minimal data requirements and increasing sophistication as needed.